Kalman Filter

Prof. Ziyun (Claude) Wang
Department of Electrical and Computer Engineering
Spring 2026

Logistics

- HW2 due next week
- Guest lecture on tactile on Wednesday
- ~04/06 Final project proposal presentation
  - Survey coming soon

State Estimation with Bayes Filter

State estimation in robot perception

Robot Localization

Object Tracking

What is state estimation in robotics?

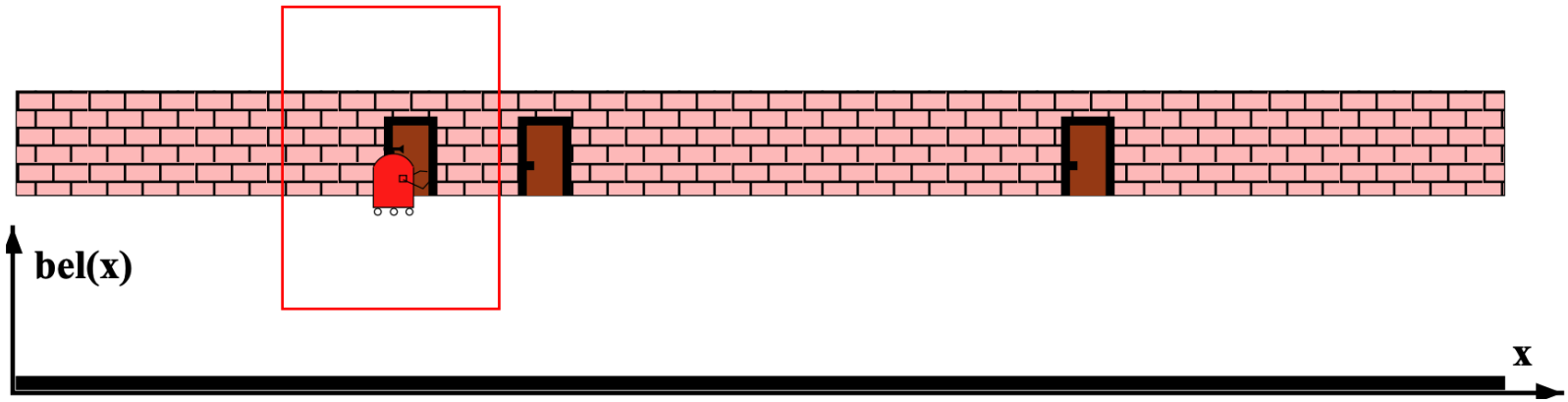
- A state is the collection of all aspects of the robot and its environment that can impact the future.
- We want to get our best estimate of the state.

Examples of objects in a state

- Robot pose, joint angles, velocities, accelerations
- Point locations from the environment

Example: global localization of a mobile robot

"Where is the robot?"



The robot knows:

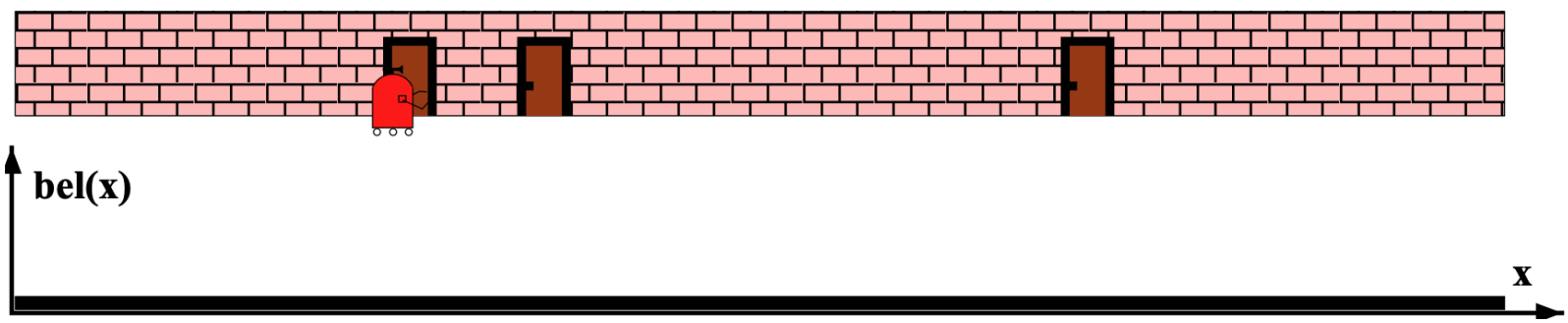
- Where the three doors are in 3D
- when it sees a door

It does not know its own location

Example: global localization of a mobile robot

bel(x) is a distribution of where the robot is

Uniform belief, no prior knowledge of the location.

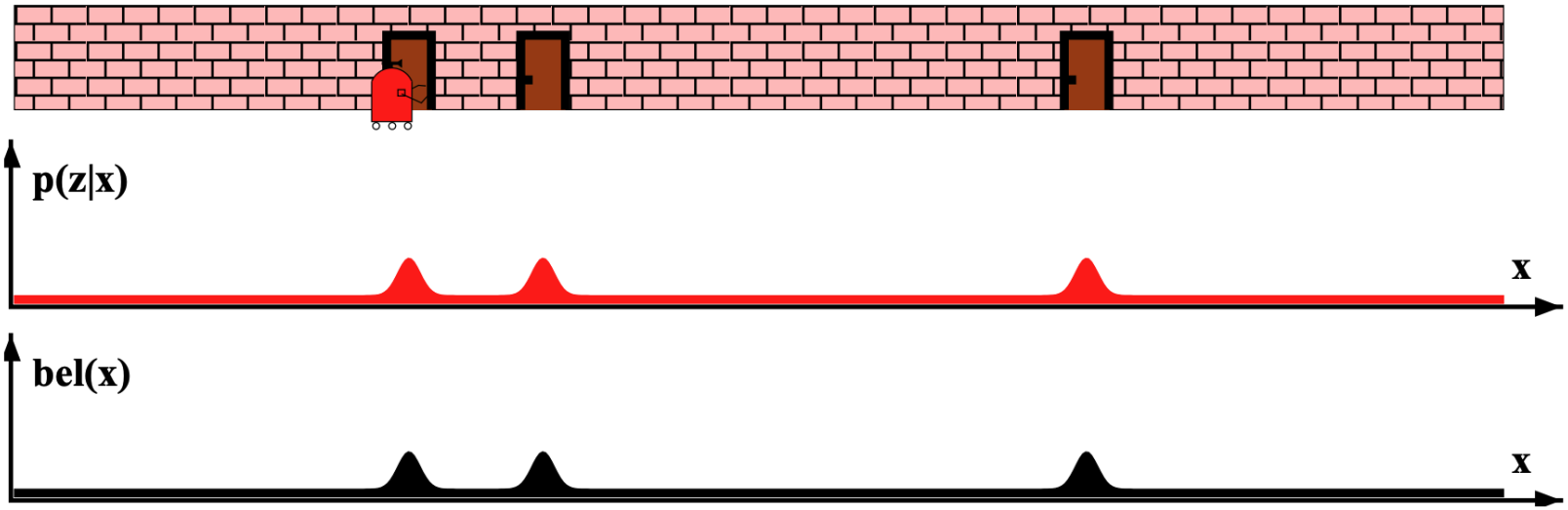


Example: global localization of a mobile robot

The robot says: “I see a door!”

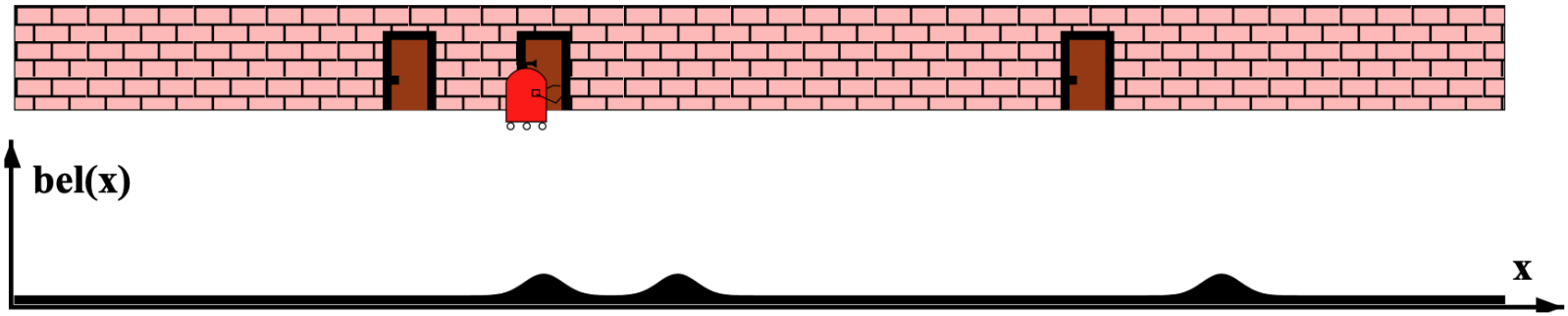
P(z|x) is the measurement likelihood

- “if the robot is at x, how likely is it to observe sensor reading z?”



Example: global localization of a mobile robot

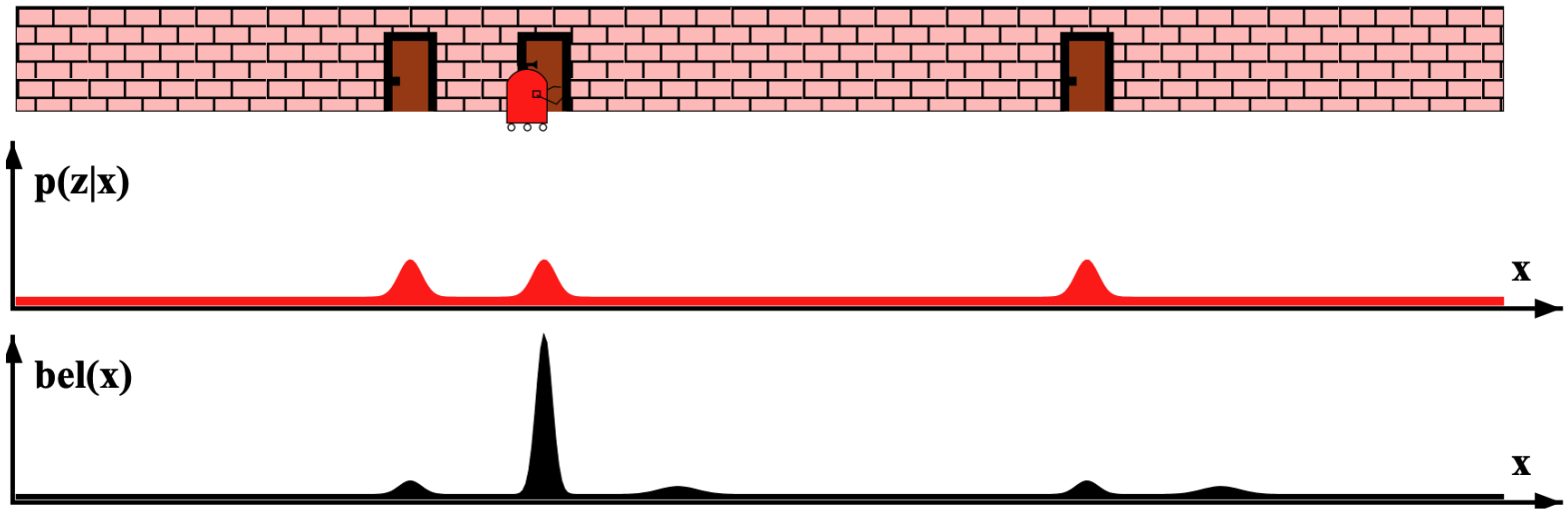
The robot moves again.



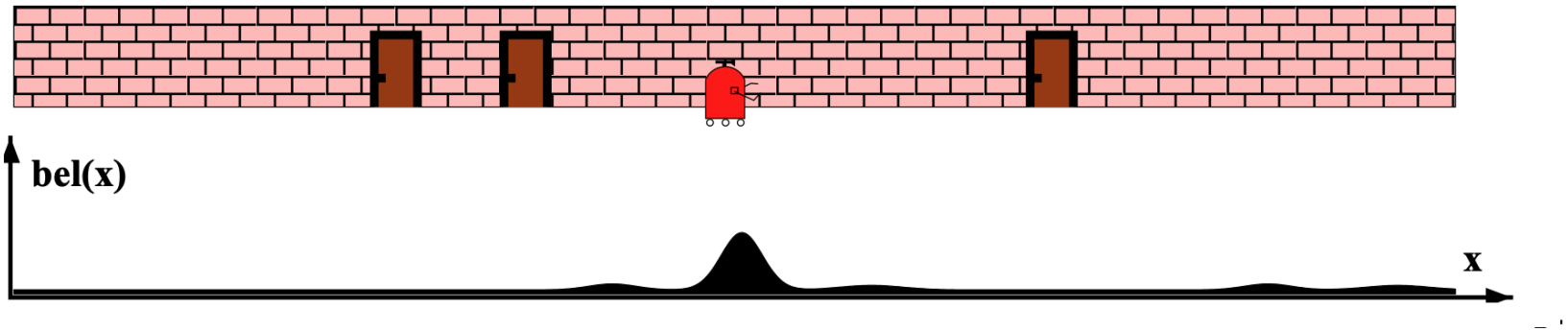
Example: global localization of a mobile robot

The robot says: “I see another door”

Now the robot is quite certain where it is



Example: global localization of a mobile robot



This is called Bayes' filter

Quick recap: what have we done?

- Have our best guess with uncertainty
- Move the robot with a motion model
- Update a new guess that combines motion and sensing
- Update the uncertainty of our estimate

Bayes filter

Bayes Rule

Let x be a hidden **state** and z be a **measurement**.

Bayes' rule

$$p(x \mid z) = \frac{p(z \mid x)\, p(x)}{p(z)}$$

- $p(x)$ **prior**: what you believed before seeing z
- $p(z \mid x)$ **likelihood**: how probable this measurement is if the state were x (sensor model)
- $p(z)$ **evidence**: average likelihood under the prior
- $p(x \mid z)$ **posterior**: updated belief after seeing z

$$\boxed{\text{posterior} \;\propto\; \text{likelihood} \times \text{prior}.}$$

Bayes rule in filter form

In a filter, the **prior** at time k is the *predicted belief* given past measurements.

Bayes' rule (conditioning on history)

$$p(x_k \mid z_{1:k}) = \frac{p(z_k \mid x_k)\, p(x_k \mid z_{1:k-1})}{p(z_k \mid z_{1:k-1})}$$

- $p(x_k \mid z_{1:k-1})$: **predicted belief** (what you believed before seeing z_k)
- $p(z_k \mid x_k)$: **likelihood** (sensor model)
- $p(z_k \mid z_{1:k-1})$: **normalizer** (evidence), $\int p(z_k \mid x_k)\, p(x_k \mid z_{1:k-1})\, dx_k$
- $p(x_k \mid z_{1:k})$: **posterior belief** after incorporating z_k

General Bayes filter recursion

Prediction

Before seeing the sensor reading z_k

$$p(x_k \mid z_{1:k-1}, u_{1:k}) = \int p(x_k \mid x_{0:k-1}, z_{1:k-1}, u_{1:k})\, p(x_{0:k-1} \mid z_{1:k-1}, u_{1:k-1})\, dx_{0:k-1}$$

Update

Correction based on sensor reading

$$p(x_k \mid z_{1:k}, u_{1:k}) = \eta\, p(z_k \mid x_k, z_{1:k-1}, u_{1:k})\, p(x_k \mid z_{1:k-1}, u_{1:k})$$

Markov assumption

Process model:

$$p(x_k \mid z_{1:k-1}, u_{1:k}) = \int p(x_k \mid x_{0:k-1}, z_{1:k-1}, u_{1:k})\; p(x_{0:k-1} \mid z_{1:k-1}, u_{1:k-1})\; dx_{0:k-1}$$

$$\Downarrow$$

$$p(x_k \mid z_{1:k-1}, u_{1:k}) = \int p(x_k \mid x_{k-1}, u_k)\; p(x_{k-1} \mid z_{1:k-1}, u_{1:k-1})\; dx_{k-1}$$

Measurement model:

$$p(z_k \mid x_k, z_{1:k-1}, u_{1:k})$$

$$\Downarrow$$

$$p(z_k \mid x_k)$$

Bayes' Filter Problem

Requires Three Ingredients

- Prior: $p(x_0)$
- Process model: $p(x_k \mid x_{k-1}, u_k)$
- Measurement model: $p(z_k \mid x_k)$

Process model tells us how we transition between states across time

Measurement model tells us how a state "maps to" sensor reading

Update the belief in two steps:

- Prediction step:

$$p(x_k \mid z_{1:k-1}, u_{1:k}) = \int p(x_k \mid x_{k-1}, u_k)\; p(x_{k-1} \mid z_{1:k-1}, u_{1:k-1})\; dx_{k-1}$$

- Update step:

$$p(x_k \mid z_{1:k}, u_{1:k}) = \eta\, p(z_k \mid x_k)\; p(x_k \mid z_{1:k-1}, u_{1:k})$$

Bayes filter algorithm

Inputs

Prior $p(x_0)$; models $p(x_k \mid x_{k-1}, u_k)$ and $p(z_k \mid x_k)$; sequence of observations $\{z_k\}$ (and optional controls $\{u_k\}$).

Recursion (for $k = 1, 2, \ldots$)

1. **Prediction**

$$\overline{\mathrm{bel}}(x_k) \leftarrow \int p(x_k \mid x_{k-1}, u_k)\ \mathrm{bel}(x_{k-1})\ dx_{k-1}$$

2. **Update (measurement correction)**

$$\mathrm{bel}(x_k) \leftarrow \eta\, p(z_k \mid x_k)\ \overline{\mathrm{bel}}(x_k)$$

3. **Normalize**

$$\eta^{-1} \leftarrow \int p(z_k \mid x_k)\ \overline{\mathrm{bel}}(x_k)\ dx_k$$

The goal is to estimate the density of the belief over time.

What do we learn so far?

- A robot carries its own uncertainty (distribution), not a single guess
- Two iterative steps:
  - Process model (moving the robot) — Propagate the belief
  - Measurement model (seeing a door) — Correct the belief

How do we implement a Bayes Filter

- We need a way to move and reshape distribution $p(x_k \mid x_{k-1}, u_k)$
- We want to consider noise in the process model and the measurement model

Kalman Filter
(where everything is Gaussian)

What is KF in the language of Bayes filter?

- Makes assumptions of the parametric distribution of each term
- And derive how we update the belief distribution

- Prediction step:

$$p(x_k \mid z_{1:k-1}, u_{1:k}) = \int p(x_k \mid x_{k-1}, u_k)\; p(x_{k-1} \mid z_{1:k-1}, u_{1:k-1})\; dx_{k-1}$$

- Update step:

$$p(x_k \mid z_{1:k}, u_{1:k}) = \eta\, p(z_k \mid x_k)\; p(x_k \mid z_{1:k-1}, u_{1:k})$$

Assumption 1: Initial state is Gaussian

Gaussian prior assumption

$$p(x_0) = \mathcal{N}(x_0\,;\,\mu_0, \mathbf{P}_0).$$

Assumption 2: motion model is Gaussian

$$p(x_k \mid x_{k-1}, u_k) = \mathcal{N}(x_k\,;\, \mathbf{F}x_{k-1} + \mathbf{B}u_k, \mathbf{Q})$$

Additive noise

Process model

$$\boldsymbol{x}_k = \mathbf{F}_k\,\boldsymbol{x}_{k-1} + \mathbf{B}_k\,\boldsymbol{u}_k + \boldsymbol{w}_k, \qquad \boldsymbol{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k)$$

- $\mathbf{F}_k$: how the state evolves (local linear dynamics)
- $\mathbf{Q}_k$: how much uncertainty we add each step

Assumption 3: measurement is Gaussian

Measurement equation (linear case) Additive noise

$$\boldsymbol{z}_k = \mathbf{H}\,\boldsymbol{x}_k + \boldsymbol{v}_k, \qquad \boldsymbol{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}).$$

Equivalent likelihood

$$p(\boldsymbol{z}_k \mid \boldsymbol{x}_k) = \mathcal{N}(\boldsymbol{z}_k\,;\,\mathbf{H}\boldsymbol{x}_k, \mathbf{R}).$$

- $\mathbf{H}$: maps state $\boldsymbol{x}_k \in \mathbb{R}^n$ to the measurement space $\boldsymbol{z}_k \in \mathbb{R}^m$
- $\boldsymbol{v}_k$: measurement noise (zero-mean random error)
- $\mathbf{R}$: measurement noise covariance (sets how much we trust the sensor)

Kalman filter

Start from the Bayes filter prediction integral:

$$p(\boldsymbol{x}_k \mid \boldsymbol{z}_{1:k-1}, \boldsymbol{u}_{1:k}) = \int p(\boldsymbol{x}_k \mid \boldsymbol{x}_{k-1}, \boldsymbol{u}_k)\; p(\boldsymbol{x}_{k-1} \mid \boldsymbol{z}_{1:k-1}, \boldsymbol{u}_{1:k-1})\; d\boldsymbol{x}_{k-1}.$$

$$p(\boldsymbol{x}_{k-1} \mid \boldsymbol{z}_{1:k-1}, \boldsymbol{u}_{1:k-1}) = \mathcal{N}(\mu_{k-1}, \mathbf{P}_{k-1})$$

$$p(\boldsymbol{x}_k \mid \boldsymbol{x}_{k-1}, \boldsymbol{u}_k) = \mathcal{N}(\mathbf{F}_k \boldsymbol{x}_{k-1} + \mathbf{B}_k \boldsymbol{u}_k, \mathbf{Q}_k).$$

Due the nice property of Gaussians under linear transformations

The belief is always Gaussian

Prediction step:

$$p(x_k \mid z_{1:k-1}, u_{1:k}) = \int p(x_k \mid x_{k-1}, u_k)\; p(x_{k-1} \mid z_{1:k-1}, u_{1:k-1})\; dx_{k-1}$$

Update step:

$$p(x_k \mid z_{1:k}, u_{1:k}) = \eta\, p(z_k \mid x_k)\; p(x_k \mid z_{1:k-1}, u_{1:k})$$

Gaussian closure through the Bayes filter steps

- **Prediction:** linear transform + additive Gaussian noise keeps a Gaussian Gaussian.
- **Update:** Gaussian likelihood $p(z_k \mid x_k)$ times predicted belief $p(x_k \mid z_{1:k-1}, u_{1:k})$ gives a Gaussian posterior.
- Therefore the belief can be summarized by $(\mu_k, \mathbf{P}_k)$ (and predicted $(\bar{\mu}_k, \bar{\mathbf{P}}_k)$).

Step 1: process model

Mean and covariance propagation

$$\bar{\mu}_k = \mathbf{F}_k \mu_{k-1} + \mathbf{B}_k \boldsymbol{u}_k$$
$$\bar{\mathbf{P}}_k = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}_k$$

Notation & intuition (prediction)

- $\bar{\mu}_k, \bar{\mathbf{P}}_k$: **predicted** mean/covariance (before seeing $\boldsymbol{z}_k$)
- $\mathbf{F}_k$: state transition (linear dynamics); $\mathbf{B}_k \boldsymbol{u}_k$: known control effect
- $\mathbf{Q}_k$: process noise covariance (unmodeled dynamics) $\Rightarrow$ increases uncertainty

Step 2: measurement step (residual)

Measurement update is just Bayes rule:

$$p(\boldsymbol{x}_k \mid \boldsymbol{z}_{1:k}, \boldsymbol{u}_{1:k}) \propto p(\boldsymbol{z}_k \mid \boldsymbol{x}_k)\, p(\boldsymbol{x}_k \mid \boldsymbol{z}_{1:k-1}, \boldsymbol{u}_{1:k}).$$

With a Gaussian predicted belief and linear-Gaussian measurement model $\boldsymbol{z}_k = \mathbf{H}_k \boldsymbol{x}_k + \boldsymbol{v}_k,\ \boldsymbol{v}_k \sim \mathcal{N}(0, \mathbf{R}_k)$, the posterior remains Gaussian.

Innovation (residual):

$$\tilde{\boldsymbol{z}}_k = \boldsymbol{z}_k - \mathbf{H}_k\, \bar{\mu}_k$$

"Error" in measurement

Step 2: measurement step (update)

Innovation covariance and Kalman gain

$$\mathbf{S}_k = \mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{R}_k$$
$$\mathbf{K}_k = \bar{\mathbf{P}}_k \mathbf{H}_k^\top \mathbf{S}_k^{-1}$$

Interpretation: $\mathbf{K}_k$ balances **model uncertainty** $(\bar{\mathbf{P}}_k)$ vs **sensor noise** $(\mathbf{R}_k)$.

The filter uses it directly to correct the state estimate:

$$\mu_k = \bar{\mu}_k + \mathbf{K}_k \tilde{z}_k.$$

$$\mathbf{P}_k = \bar{\mathbf{P}}_k - \mathbf{K}_k \mathbf{H}_k \bar{\mathbf{P}}_k.$$

KF update is just conditioning a joint Gaussian

Start from the predicted belief and linear-Gaussian sensor:

$$\boldsymbol{x}_k \sim \mathcal{N}(\bar{\mu}_k, \bar{\mathbf{P}}_k), \qquad \boldsymbol{z}_k = \mathbf{H}_k \boldsymbol{x}_k + \boldsymbol{v}_k,\ \boldsymbol{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k).$$

Then $\begin{bmatrix} \boldsymbol{x}_k \\ \boldsymbol{z}_k \end{bmatrix}$ is jointly Gaussian with

$$\begin{bmatrix} \boldsymbol{x}_k \\ \boldsymbol{z}_k \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \bar{\mu}_k \\ \mathbf{H}_k \bar{\mu}_k \end{bmatrix}, \begin{bmatrix} \bar{\mathbf{P}}_k & \bar{\mathbf{P}}_k \mathbf{H}_k^\top \\ \mathbf{H}_k \bar{\mathbf{P}}_k & \mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{R}_k \end{bmatrix} \right).$$

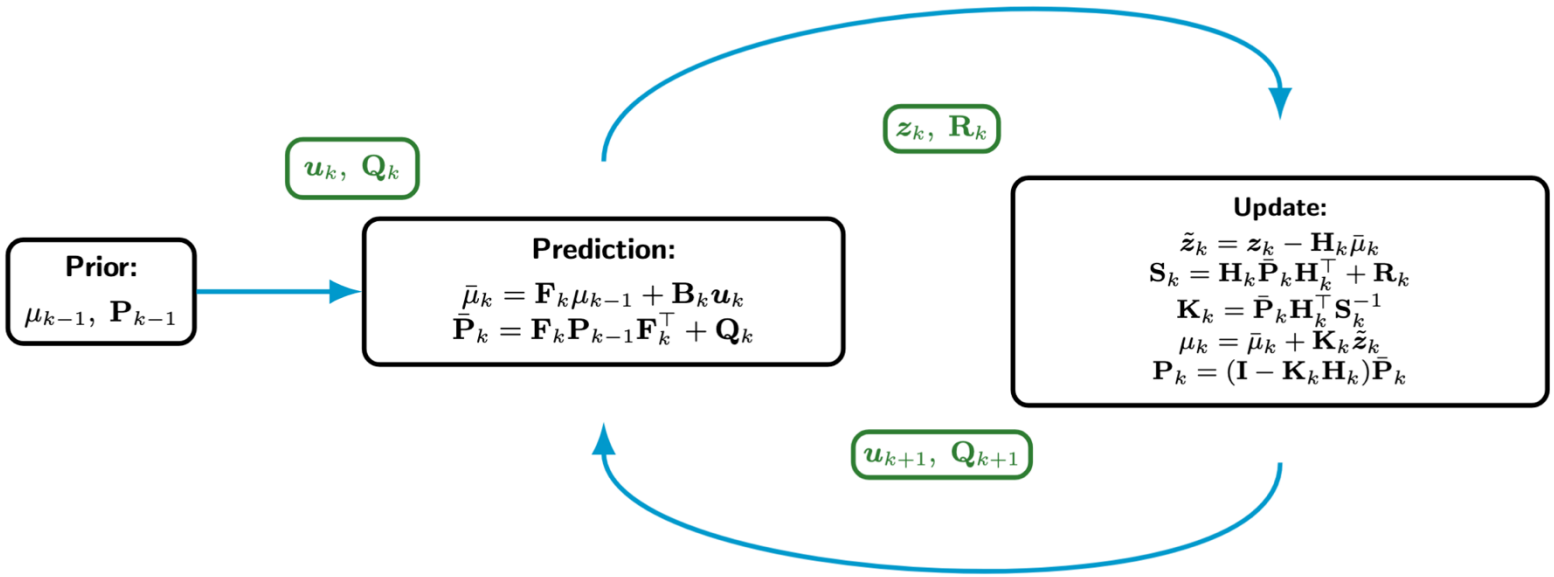
Key idea: the KF measurement update is the conditional Gaussian $p(\boldsymbol{x}_k \mid \boldsymbol{z}_k)$ from this joint.

KF update is just conditioning a joint Gaussian

From the standard conditional-Gaussian formula (conditioning the joint):

$$
\begin{aligned}
\text{innovation } \boldsymbol{r}_k &= \boldsymbol{z}_k - \mathbf{H}_k \bar{\mu}_k, \\
\text{innovation cov. } \mathbf{S}_k &= \mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{R}_k, \\
\text{gain } \mathbf{K}_k &= \bar{\mathbf{P}}_k \mathbf{H}_k^\top \mathbf{S}_k^{-1}, \\
\text{mean update } \mu_k &= \bar{\mu}_k + \mathbf{K}_k \boldsymbol{r}_k, \\
\text{cov update } \mathbf{P}_k &= \bar{\mathbf{P}}_k - \mathbf{K}_k \mathbf{S}_k \mathbf{K}_k^\top.
\end{aligned}
$$

Kalman filter iterations



Full steps

Given at time k

- **From previous iteration:** μ_{k-1}, $\mathbf{P}_{k-1}$
- **Models / noise stats (fixed or known):** $\mathbf{F}_k$, $\mathbf{Q}_k$, $\mathbf{H}_k$, $\mathbf{R}_k$
- **New data at time k:** $\boldsymbol{z}_k$ (and optional control $\boldsymbol{u}_k$)

Compute at this time step (then discard intermediates)

Predict

$$\bar{\mu}_k = \mathbf{F}_k \mu_{k-1} + \mathbf{B}_k \boldsymbol{u}_k,$$
$$\bar{\mathbf{P}}_k = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}_k.$$

Update

$$\mathbf{S}_k = \mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{R}_k,$$
$$\mathbf{K}_k = \bar{\mathbf{P}}_k \mathbf{H}_k^\top \mathbf{S}_k^{-1}.$$
$$\mu_k = \bar{\mu}_k + \mathbf{K}_k \left(\boldsymbol{z}_k - \mathbf{H}_k \bar{\mu}_k \right),$$
$$\mathbf{P}_k = \bar{\mathbf{P}}_k - \mathbf{K}_k \mathbf{H}_k \bar{\mathbf{P}}_k.$$

Keep for the next iteration $(k+1)$

Store only the posterior: μ_k, $\mathbf{P}_k$. These become μ_k, $\mathbf{P}_k$ at the next step.

Intuition 1: move and reshape of a belief Gaussian

- Process model moves and stretches the Gaussian
- The measurement model ”corrects” the mean and covariance based on the sensors

Intuition 2: balance "trust"

- "Kalman gain" is how much we trust the sensor vs. the prediction

$$\mathbf{K}_k = \bar{\mathbf{P}}_k \mathbf{H}_k^\top \left(\mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{R}_k\right)^{-1}$$

- **Perfect sensor** $\mathbf{R}_k = \mathbf{0}$ (noise-free measurement)
  - $\mathbf{K}_k = \bar{\mathbf{P}}_k \mathbf{H}_k^\top (\mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{R}_k)^{-1} = \bar{\mathbf{P}}_k \mathbf{H}_k^\top (\mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top)^{-1}$
  - $\mu_k = \bar{\mu}_k + \mathbf{K}_k \left(\boldsymbol{z}_k - \mathbf{H}_k \bar{\mu}_k\right)$
  - $\mathbf{P}_k = \bar{\mathbf{P}}_k - \mathbf{K}_k \mathbf{H}_k \bar{\mathbf{P}}_k$

- **Horrible sensor** $\mathbf{R}_k \to \infty$
  - $\mathbf{K}_k = \bar{\mathbf{P}}_k \mathbf{H}_k^\top (\mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{R}_k)^{-1} \to \mathbf{0}$
  - $\mu_k = \bar{\mu}_k + \mathbf{K}_k \left(\boldsymbol{z}_k - \mathbf{H}_k \bar{\mu}_k\right) \to \bar{\mu}_k$
  - $\mathbf{P}_k = \bar{\mathbf{P}}_k - \mathbf{K}_k \mathbf{H}_k \bar{\mathbf{P}}_k \to \bar{\mathbf{P}}_k$

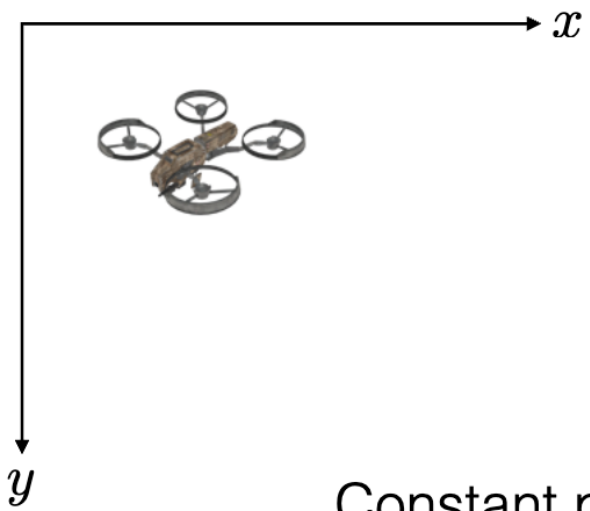
How errors affect the correction

The gain depends on predicted uncertainty and measurement noise:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\,\mathbf{H}_k^\top\,(\mathbf{H}_k\,\mathbf{P}_{k|k-1}\,\mathbf{H}_k^\top + \mathbf{R}_k)^{-1}.$$

- Larger $\mathbf{R}_k$ (noisier sensor) $\Rightarrow$ smaller $\mathbf{K}_k$ $\Rightarrow$ trust the prediction more.
- Larger $\mathbf{P}_{k|k-1}$ (more uncertain prediction) $\Rightarrow$ larger $\mathbf{K}_k$ $\Rightarrow$ trust the measurement more.
- Larger $\mathbf{Q}_k$ increases $\mathbf{P}_{k|k-1}$ during prediction $\Rightarrow$ also increases how much future measurements can correct.

Example: 2D robot



state

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

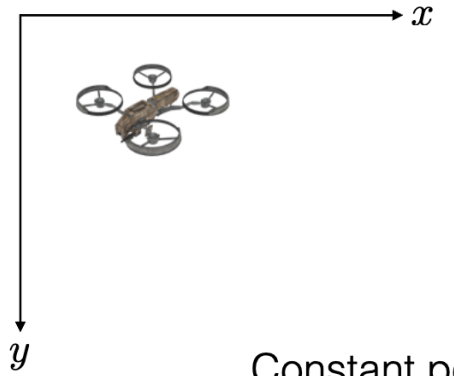
measurement

$$\boldsymbol{z} = \begin{bmatrix} x \\ y \end{bmatrix}$$

Constant position Motion Model

$$\boldsymbol{x}_t = A\boldsymbol{x}_{t-1} + B\boldsymbol{u}_t + \epsilon_t$$

Example: 2D robot



state

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

measurement

$$\boldsymbol{z} = \begin{bmatrix} x \\ y \end{bmatrix}$$

Constant position Motion Model

$$\boldsymbol{x}_t = A\boldsymbol{x}_{t-1} + B\boldsymbol{u}_t + \epsilon_t$$

Constant position

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

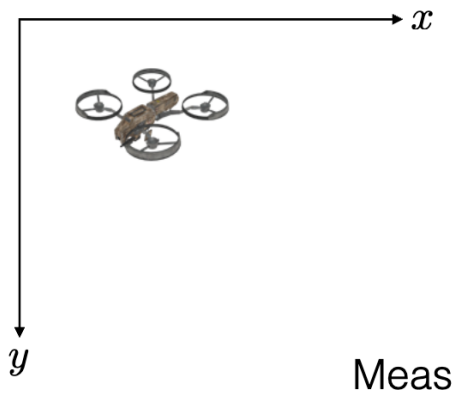
$$B\boldsymbol{u} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

system noise

$$\epsilon_t \sim \mathcal{N}(\mathbf{0}, R)$$

$$R = \begin{bmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_r^2 \end{bmatrix}$$

Example: 2D robot



state

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

measurement

$$\boldsymbol{z} = \begin{bmatrix} x \\ y \end{bmatrix}$$

Measurement Model

$$\boldsymbol{z}_t = C_t \boldsymbol{x}_t + \delta_t$$

zero-mean measurement noise

$$C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \delta_t \sim \mathcal{N}(\mathbf{0}, Q) \qquad Q = \begin{bmatrix} \sigma_q^2 & 0 \\ 0 & \sigma_q^2 \end{bmatrix}$$

Example: 2D robot

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

motion model — observation model

General Case

$$\bar{\mu}_t = A_t \mu_{t-1} + B u_t$$
$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^\top + R$$
$$K_t = \bar{\Sigma}_t C_t^\top (C_t \bar{\Sigma}_t C_t^\top + Q_t)^{-1}$$
$$\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$$
$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$$

Constant position Model

$$\bar{\boldsymbol{x}}_t = \boldsymbol{x}_{t-1}$$
$$\bar{\Sigma}_t = \Sigma_{t-1} + R$$
$$K_t = \bar{\Sigma}_t (\bar{\Sigma}_t + Q)^{-1}$$
$$\boldsymbol{x}_t = \bar{\boldsymbol{x}}_t + K_t (z_t - \bar{\boldsymbol{x}}_t)$$
$$\Sigma_t = (I - K_t) \bar{\Sigma}_t$$

Example 2D tracking

State: $\boldsymbol{x}_k = \begin{bmatrix} p_{x,k} & p_{y,k} & v_{x,k} & v_{y,k} \end{bmatrix}^\top \in \mathbb{R}^4$, $p(\boldsymbol{x}_k \mid \boldsymbol{z}_{1:k}) = \mathcal{N}(\mu_k, \mathbf{P}_k)$.

Process (constant velocity + unknown acceleration):

$$\boldsymbol{x}_k = \mathbf{F}_k \boldsymbol{x}_{k-1} + \boldsymbol{w}_k, \qquad \boldsymbol{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k), \qquad \mathbf{F}_k = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Measurement (detector returns 2D position):

$$\boldsymbol{z}_k = \mathbf{H}_k \boldsymbol{x}_k + \boldsymbol{v}_k, \qquad \boldsymbol{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k), \qquad \mathbf{H}_k = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$